Experiment No. 2

Design Patterns Implementation – Implement and Compare Different Design Patterns

Aim

To understand, implement, and compare different software design patterns by developing programs using Creational, Structural, and Behavioral design patterns and analyzing their advantages, disadvantages, and applications.

Objectives

After completing this experiment, students will be able to:

- Understand the concept of software design patterns.
- Differentiate among Creational, Structural, and Behavioral design patterns.
- Implement commonly used design patterns in Java (or any OOP language).
- Compare different design patterns based on their implementation and usage.
- Improve software reusability, maintainability, and flexibility.

Theory

What are Design Patterns?

A **Design Pattern** is a reusable solution to commonly occurring software design problems. It is not a complete program but a template describing how to solve a recurring design problem.

Design patterns provide proven solutions that make software easier to understand, modify, and maintain.

Benefits

- Reusable solutions
- Better software architecture
- Improved code readability
- Easier maintenance
- Reduced development time
- Promotes loose coupling

Categories of Design Patterns

Category	Purpose	Examples
Creational	Object creation	Singleton, Factory, Builder
Structural	Object composition	Adapter, Decorator, Facade
Behavioral	Object interaction	Observer, Strategy, Command

Software Requirements

- Java JDK 17 or above
- Eclipse IDE / IntelliJ IDEA / NetBeans
- Windows/Linux
- Command Prompt or Terminal

Hardware Requirements

- Intel i3 or higher processor
- 4 GB RAM (Minimum)
- 10 GB Free Disk Space

Problem Statement

Implement the following design patterns:

1. Singleton Pattern
2. Factory Pattern
3. Observer Pattern

Execute each implementation and compare them based on:

- Purpose
- Complexity
- Coupling
- Flexibility
- Real-world applications

Algorithms

A. Singleton Pattern

1. Create a class.
2. Make constructor private.
3. Create static instance.
4. Provide public static method.
5. Return same object every time.

B. Factory Pattern

1. Create interface.
2. Create multiple implementations.
3. Create Factory class.
4. Return object based on user input.
5. Execute required method.

C. Observer Pattern

1. Create Subject.
2. Register observers.
3. Notify observers whenever state changes.
4. Display updated values.

Program

Program 1: Singleton Pattern

```
class Singleton {
    private static Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if(instance == null)
            instance = new Singleton();
        return instance;
    }

    public void display() {
        System.out.println("Singleton Object Created");
    }
}

public class SingletonDemo {
    public static void main(String args[]) {

        Singleton s1 = Singleton.getInstance();
        Singleton s2 = Singleton.getInstance();

        s1.display();

        System.out.println(s1 == s2);
    }
}
```

Output

```
Singleton Object Created
true
```

Comparison of Design Patterns

Feature	Singleton	Factory	Observer
Category	Creational	Creational	Behavioral
Purpose	One object only	Object creation	Notification mechanism
Coupling	Low	Low	Very Low
Flexibility	Medium	High	High
Complexity	Low	Medium	Medium
Reusability	High	High	High
Best Used	Database connection, Logger	Object creation	Event handling